

Programs you don't write

Loss, gradients, and the machine that turns data into functions

You have written a thousand functions in your career. Here is one you cannot write:

`is_spam(email) -> bool`. You can try. You will start with keyword rules ("free", "winner", "click here"), then add sender reputation, then exceptions for your colleague who genuinely writes like a spammer, and six months later you will own two thousand lines of special cases that spammers route around in a weekend. Now consider a much harder function:

`next_word("The capital of Peru is") -> str`. Nobody on Earth can write that one by hand, and yet you have used it today. Llama 2 7B is exactly that function: a single, fixed piece of arithmetic with 6,738,415,616 constants in it, and not one of those constants was chosen by a person.¹

This chapter is about the trick that makes such functions possible, because everything else in this book — the GPUs, the parallelism, the data pipelines, the hundred-million-dollar training runs — exists to execute this one trick at scale. The trick is: stop writing the function, and instead write a *score* for how wrong a candidate function is, then let an optimization algorithm search for the candidate that scores best. You write the loss; the loss writes the program. If you are a software engineer, this is the single largest mental shift in the field, bigger than anything about transformers, and once it clicks, the rest of the book is engineering.

We will build the whole recipe on an example small enough to compute by hand, watch PyTorch reproduce our hand computation exactly, see why the recipe fails silently when you evaluate it wrong, and finish with why all of this runs on GPUs. Nothing in this chapter is specific to language models; Chapter 2 will swap in text, and nothing we build here will change.

1.1 A function you can't write by hand

Think about why `is_spam` defeated you. It was not lack of skill. The function you were trying to express is a boundary through a space with millions of dimensions — every word that could appear, every combination, every tone — and the boundary is ragged, full of exceptions, and moving. Code is a terrible medium for ragged boundaries. Every `if` you write is a flat wall, and you would need millions of walls, positioned by someone who could see the whole space at once. Nobody can.

But here is what you *do* have: examples. Fifty thousand emails your users already flagged, and fifty thousand they did not. Each example is a point on the correct side of the boundary. The examples know things about the boundary that you do not, and the machine learning move is to transfer that knowledge from the examples into a function

automatically. The pieces are three. First, a *family* of candidate functions, flexible enough to express ragged boundaries, written as one formula with free constants in it — the *parameters*. Pick different constants, get a different function. Second, a *loss*: a single number measuring how badly a particular choice of constants disagrees with your examples. Third, an *optimizer*: a procedure that adjusts the constants to make the loss smaller. Run the optimizer long enough and the constants encode what the examples knew.

The way you know

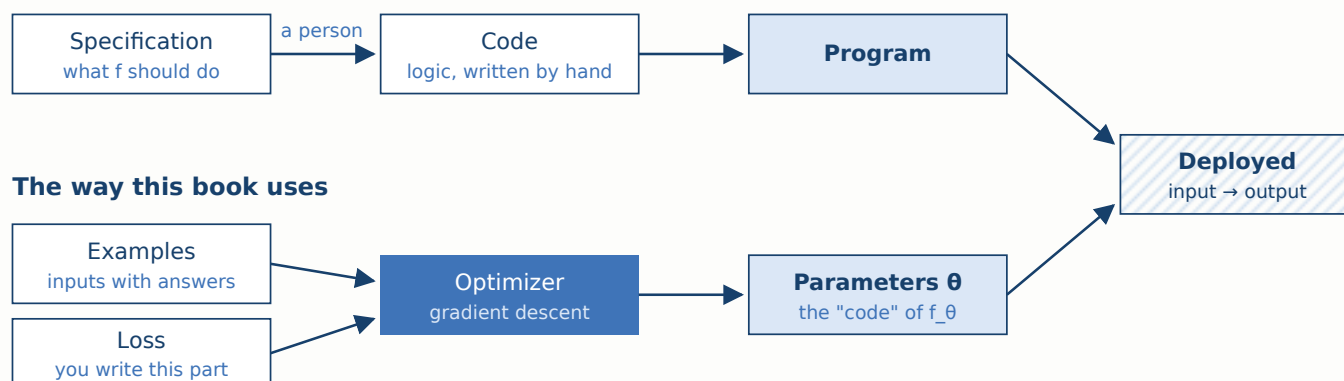


Figure 1.1 — Two ways to produce a program. Classical engineering turns a specification into logic by hand. Machine learning turns examples plus a loss into parameters by optimization. Both end in a deployed function; the difference is who wrote its internals. In this book the "program" is a transformer and θ has billions of entries, but this picture does not change.

Notice what your job becomes under this contract. You no longer write the logic; you choose the function family (that is *architecture*, Chapter 3), you assemble the examples (that is *data*, Chapter 10 — and it is the most underrated job in the field), you define the loss (for language models, Chapter 2), and you run the optimizer, which for models of interesting size means the distributed systems problem that fills Part III of this book. The skills transfer: what used to be debugging logic becomes debugging data, loss curves, and gradients. The rest of this chapter builds each piece on the smallest example that is not a toy in spirit: two parameters instead of six billion, but the very same recipe, and — this matters — the very same failure modes.

1.2 Parameters, loss, and the gradient

Say you are estimating apartment rents. You have three data points from listings: 50 m² rents for \$1,550, 80 m² for \$2,350, 120 m² for \$3,700. You suspect rent is roughly proportional to area, so you pick the simplest family of functions that can express that:

$$f_{\theta}(x) = w x + b, \quad \theta = (w, b).$$

Two parameters. The slope w is dollars per square meter and the intercept b is a base cost. Every choice of (w, b) is a different candidate function, and most candidates are terrible: $w = 0$ predicts every apartment costs the same; $w = 500$ predicts a small fortune for a studio. We need to measure *how* terrible, and the standard measure for numeric predictions is the mean squared error: run the candidate on every example, take how far each prediction landed from the truth, square it, average:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2.$$

The squaring does two jobs: it makes undershooting and overshooting equally bad, and it punishes large misses much more than small ones. For the candidate $w = 0, b = 0$ (predict zero for everything), our three examples give $L = (1550^2 + 2350^2 + 3700^2)/3 = 7,205,000$. That number is the whole interface between the problem and the optimizer. The optimizer will never see an apartment, a square meter, or a dollar; it sees a single scalar that it wants to push down, and this is just as true when the loss summarizes next-word prediction over trillions of tokens.

So how do you push it down? Here is the idea that powers everything. Standing at the current θ , ask: *if I nudge each parameter a tiny bit, which direction makes the loss drop fastest?* That direction is given by the gradient $\nabla L(\theta)$, the vector of partial derivatives of the loss with respect to each parameter. The gradient points uphill; so step the other way, by a small factor η called the learning rate:

$$\theta \leftarrow \theta - \eta \nabla L(\theta).$$

That single line is gradient descent, and it is the optimizer inside every training run in this book — dressed up with momentum and adaptive scaling by the time we reach AdamW in Chapter 17, but recognizably this line.² Run it on our three apartments starting from $w = 0, b = 0$ with $\eta = 5 \times 10^{-5}$, and watch the loss (we ran this; the code is below):

Table 1.1 — Gradient descent on the rent example, three data points, starting from $w = 0, b = 0$. Ten steps take the loss from 7.2 million to its floor; the remaining 3,425 is the noise in the data, which no straight line can remove.

STEP	LOSS	W (\$/M ²)	$\partial L / \partial W$
0	7,205,000	0.00	−473,000
1	362,337	23.65	−105,594
2	21,313	28.93	−23,573
10	3,426	30.45	−0.2
2,000	3,418	30.46	−0.1

Three things in that table are worth staring at, because you will see them again on a 70B run. The loss falls by orders of magnitude in the first steps and then flattens: early progress is cheap, late progress is expensive. The gradient shrinks as we approach the bottom: the slope of the bowl goes to zero at its floor, so the steps get smaller by themselves. And the loss does not reach zero: our family (straight lines) cannot represent the data perfectly because the data is noisy, so it settles at the best a line can do, $w \approx 30.45$ dollars per square meter. Asked about a 100 m² apartment the model has never seen, it says \$3,045 — a sensible interpolation the three examples never stated explicitly. That, in miniature, is the entire phenomenon of learning.

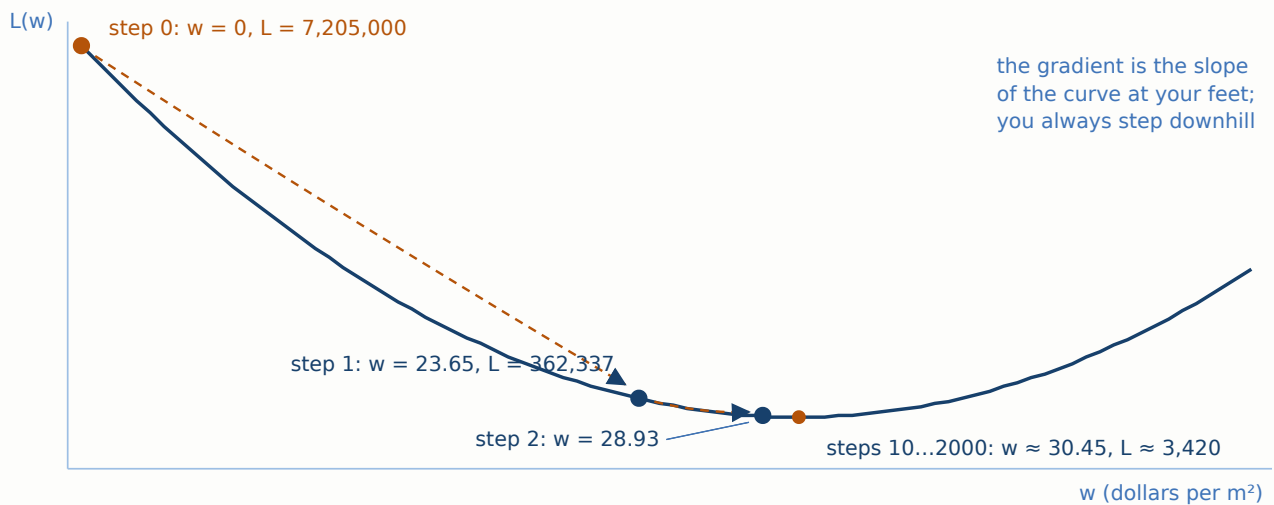


Figure 1.2 — The loss is a landscape; training is descent. The curve is the loss of the rent model as a function of the slope w (with b held at its final value). Gradient descent starts at $w = 0$, takes huge steps while the slope is steep, small ones near the floor, and settles at the best line the family can offer. With billions of parameters the landscape has billions of axes and cannot be drawn — but the local rule, "feel the slope, step downhill," is unchanged.

Here is the run, complete, in numpy. There is no framework magic below; this is everything:

```
import numpy as np

x = np.array([50., 80., 120.])      # areas (m²)
y = np.array([1550., 2350., 3700.]) # rents ($)
w, b, lr = 0.0, 0.0, 5e-5

for step in range(2001):
    pred = w * x + b                # forward: run the candidate
    err = pred - y
    loss = (err ** 2).mean()        # loss: one number
    grad_w = 2 * (err * x).mean()   # backward: d loss / d w
    grad_b = 2 * err.mean()         # backward: d loss / d b
    w -= lr * grad_w                # update: step downhill
    b -= lr * grad_b

print(w, b)                        # ≈ 30.46, ≈ -0.5
print(w * 100 + b)                 # ≈ $3,045 for an unseen 100 m² apartment
```

Read the loop body again: forward, loss, backward, update. Those four lines are a complete training run. Chapter 4 will grow this loop with batching, checkpointing, and a GPU underneath; Part III will spread it over thousands of GPUs; and at no point will it stop being these four lines. What does need to change is the middle: for two parameters we derived `grad_w` and `grad_b` on paper. Llama 2 has 6.7 billion parameters arranged in nested matrix products, softmaxes, and normalizations. Nobody derives that by hand — and nobody needs to, which is the subject of the next section.

1.3 Backpropagation: autodiff over a call graph

You already know the data structure at the heart of backpropagation; you have profiled with it. When your code runs, it traces out a call graph: this value fed that operation, whose result fed the next. A numeric program does the same, just with arithmetic: multiply, add, subtract, square. And a rule you learned in calculus — the chain rule — says that if you know how each *individual* operation stretches or shrinks changes passing through it, you can compose those local sensitivities along the graph to get the sensitivity of the final output to any input. Backpropagation is nothing more than the chain rule, executed mechanically over the program's graph, in reverse.³

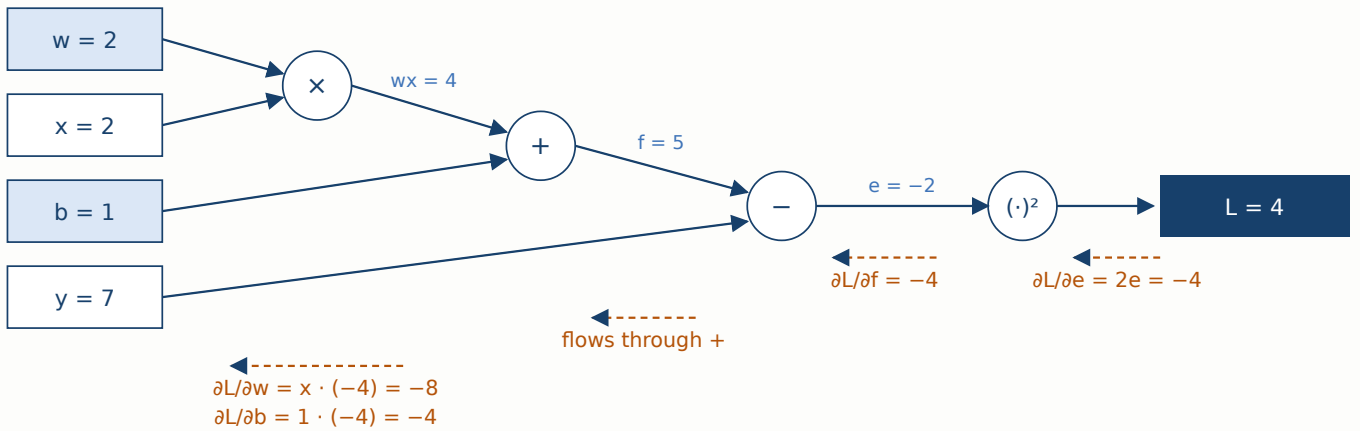
Let us do one full pass by hand on a single training example, and keep the numbers small: model $f = wx + b$ with $w = 2$, $b = 1$, input $x = 2$, target $y = 7$, and loss $L = (f - y)^2$. The forward pass computes and *records* each intermediate. The backward pass then walks the same graph in reverse, asking one question at every node — *how much does the loss change per unit change here?* — and multiplying in each operation's local derivative as it goes. Table 1.2 traces both directions; read the forward column top to bottom, then the backward column bottom to top.

Table 1.2 — One forward and backward pass, node by node. Forward computes and stores values; backward carries the sensitivity $\partial L / \partial(\cdot)$ from the loss back to the parameters, one local derivative at a time. This is the same graph as Figure 1.3.

NODE	FORWARD VALUE	BACKWARD: THE LOCAL RULE	SENSITIVITY
$L = e^2$	4	start of the walk: $\partial L / \partial L = 1$	1
$e = f - y$	-2	through the square: $\partial L / \partial e = 2e$	-4
$f = wx + b$	5	the subtraction passes it through unchanged	-4
w	2	a unit nudge of w moves f by $x = 2$, so multiply by 2	-8
b	1	a unit nudge of b moves f by 1	-4

The bottom two rows are the gradient: $(-8, -4)$. Take one descent step with $\eta = 0.1$ and the parameters move to $w = 2.8$, $b = 1.4$ — and the new prediction is $2.8 \times 2 + 1.4 = 7.0$, exactly the target. (A single example with a nearly linear model is a lucky case; real steps land closer, not exact. But it shows the machinery doing its job.)

Forward: compute and record



Backward: chain the sensitivities

Figure 1.3 — One forward and backward pass, with every number. Solid arrows: the forward computation, left to right, recording each intermediate (blue labels). Dashed amber: the backward pass, right to left, multiplying each operation's local derivative into the running sensitivity. The parameters (shaded boxes) receive their gradients last. This graph has five nodes; Llama 2's has hundreds of thousands, walked by the same two passes.

Two properties of this procedure decide much of this book's economics, so let us name them now. First, *cost*: the backward pass visits each operation about twice (once per input), so computing the full gradient costs roughly two extra forward passes, regardless of how many parameters there are. That factor is where the training rule of thumb "6 FLOPs per parameter per token" comes from — 2 for the forward, 4 for the backward — and Chapter 8 derives it properly.³ Getting six billion partial derivatives for 3× the price of one prediction is, frankly, absurd, and it is the reason deep learning works at all. Second, *memory*: the backward pass needs the recorded intermediates — that $f = 5$ and $e = -2$ — which for a transformer means every activation of every layer for every token in the batch must sit in GPU memory until the backward pass consumes it. That storage, not the parameters, is what actually fills a training GPU, and trading it against recomputation is Chapter 15.

Of course you will never write a backward pass by hand. Frameworks record the graph as your forward code executes and replay it in reverse automatically — that is all "autograd" means.⁴ Here is PyTorch reproducing our hand computation to the digit:

```
import torch

w = torch.tensor(2.0, requires_grad=True)  # "track operations on me"
b = torch.tensor(1.0, requires_grad=True)
x, y = torch.tensor(2.0), torch.tensor(7.0)

loss = (w * x + b - y) ** 2  # forward: graph recorded here
loss.backward()             # backward: chain rule, in reverse

print(loss.item())          # 4.0
print(w.grad.item())        # -8.0  ← matches our hand computation
print(b.grad.item())        # -4.0
```

The line worth internalizing is that `loss` is not just a number; it is a number that remembers how it was computed. When you later meet `with torch.no_grad():` in evaluation code, or `.detach()`, you now know exactly what they do: they stop the recording, which saves the memory and time that graph-building costs when no gradient will ever be asked for.

1.4 Train, validate, test: when "the loss went down" isn't "it works"

Now the failure mode, and it is the one that costs the most money in practice because nothing crashes. Gradient descent optimizes exactly the number you gave it: the loss *on the training examples*. It never promised anything about examples it has not seen. And with enough parameters, a model can drive the training loss down by simply *memorizing* the answers — a lookup table wearing a neural network's clothes. This is not hypothetical: a landmark experiment showed that standard image models can reach zero training error on a dataset whose labels were replaced by pure random noise, a dataset where there is, by construction, nothing to learn.⁵ The optimizer did its job perfectly; the model knows nothing. A 7B transformer has capacity to spare for this trick, and it will use it if you let it.

What you actually want is *generalization*: low loss on data the model has never seen, drawn from the same distribution. Since you cannot measure that on the training set by definition, you buy an estimate of it with data: before training, split off a slice — the *validation set* — that the optimizer never touches, and evaluate on it as you train. While the model is learning real structure, both curves fall together. When it shifts to memorizing quirks of the training slice, the training loss keeps falling and the validation loss stops — and then rises. That divergence is *overfitting*, and the validation curve is your only instrument that detects it.

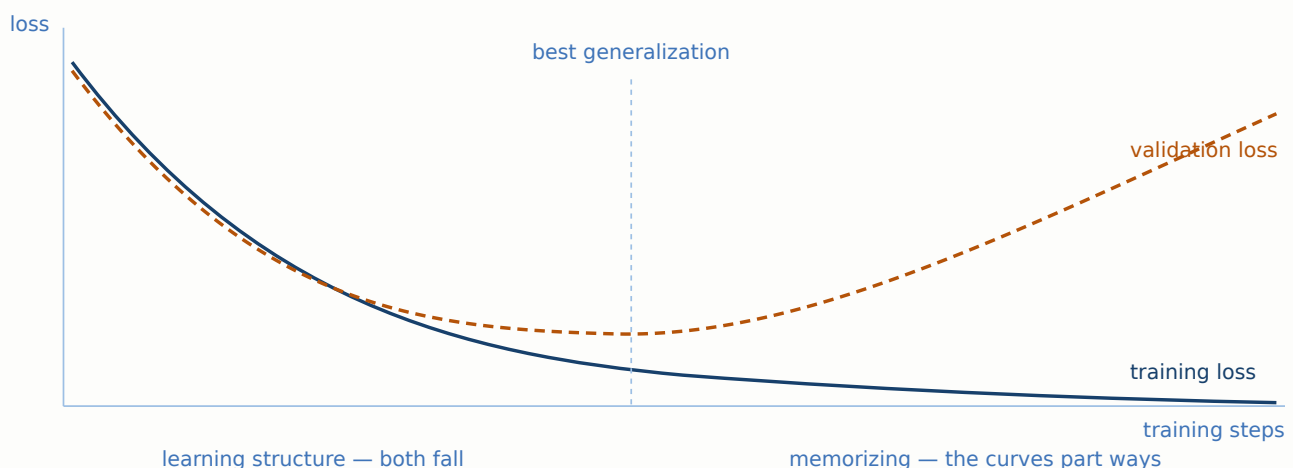


Figure 1.4 — The two curves that matter. The optimizer only ever sees the solid curve, and it will happily ride it to zero. The dashed curve, measured on held-out data, is the one that predicts how the model behaves in the world. The moment they part ways, further training makes the model worse at its actual job.

The discipline that goes with this has three tiers, and the third is the one engineers violate. The *training set* is for the optimizer. The *validation set* is for you: you look at it often, and you use it to choose learning rates, architectures, and when to stop. But every time you make a choice because validation looked better, you leak a little information about the validation set into the model — you are, slowly, hand-optimizing against it. So a final *test set* stays in a drawer,

untouched, until the work is done, and is used once. If you evaluate against it weekly, you do not have a test set; you have a second validation set and no honest estimate at all.

One adjustment for the LLM era, because the picture above was drawn for models trained many epochs over small datasets. Pre-training is usually the opposite regime: the dataset is so large that the model sees most examples about once, and classic overfitting — validation loss rising — is rare in a healthy run. Held-out loss still gets measured, and it is your best single health metric. But the failure mode mutates into something sneakier: the "unseen" evaluation may not be unseen. Web-scale training corpora frequently contain the benchmarks themselves — the MMLU questions, the HumanEval solutions — verbatim or paraphrased, and a model that memorized the answer sheet looks brilliant on it while being no better in your product. That is *contamination*, it is the industrial-scale version of testing on your training set, and Chapter 9 is largely about defending against it. The principle you carry from this section is unchanged either way: the number the optimizer minimizes is never the number you actually care about, and you must always maintain an honest, untouched way to measure the second one.

1.5 Tensors, matrix multiplies, and why GPUs exist

Everything so far was arithmetic on a handful of scalars. Scale has a shape, and the shape is the *tensor* — the n -dimensional array you know from numpy. A token's representation is a vector, shape (d) . A sequence of tokens stacks into a matrix, (L, d) . A batch of sequences stacks again, (B, L, d) . Nothing deep: tensors are how you process many examples with one operation instead of a Python loop, and the reason that matters is what the hardware does with it.

Here is the observation that explains the GPU. Take the model layer you will meet everywhere in this book, a linear layer: multiply the input by a weight matrix. For one input vector that is a matrix–vector product. For a batch of BL vectors it becomes one matrix–matrix product, $(BL, d) \times (d, d')$ — and a matrix product of shapes $(m, k) \times (k, n)$ costs almost exactly $2mkn$ floating-point operations (each output element is a length- k dot product: k multiplies, k adds). Those operations are gloriously independent: every output element can be computed at the same time as every other. An algorithm like that does not want a few fast cores that excel at branching — it wants tens of thousands of simple cores running in lockstep, which is precisely what a GPU is, plus dedicated matrix-multiply units ("tensor cores") that do nothing else.

The magnitudes deserve numbers, and we measured the first two on the machine this chapter was written on. A matrix multiply written as pure Python loops crawls at about 0.03 GFLOP/s. The same multiply through numpy — same CPU, but vectorized, cache-blocked, multi-threaded — runs at about 236 GFLOP/s: a factor of 7,000 from using the hardware properly. An A100 GPU does 312 TFLOP/s in BF16, a thousand times the CPU; an H100 does 989 TFLOP/s.⁶⁷ End to end, the ladder from naive loops to one modern training GPU spans seven orders of magnitude — and a 70B pre-training run occupies *thousands* of such GPUs for months. When you hear that training a frontier model costs 10^{25} FLOPs, this ladder is what makes the number physically possible.

Matrix-multiply throughput (log scale)

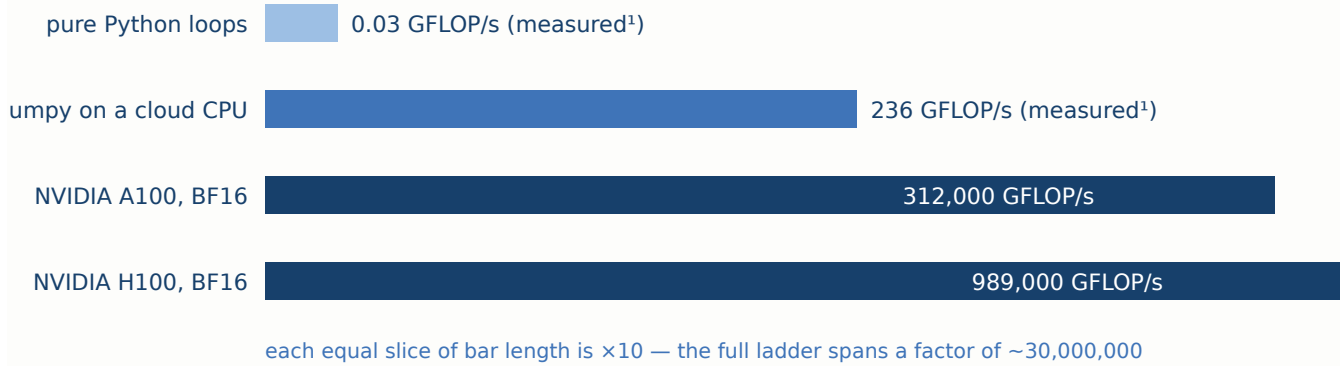


Figure 1.5 — Why GPUs exist, in one ladder. The same operation, a large matrix multiply, at four rungs of the hardware stack; bar length is logarithmic. ¹The Python and numpy figures we measured ourselves (2048² matrices, float32, single cloud CPU); the GPU figures are the vendor's peak dense BF16 numbers,⁶⁷ which real training reaches only in part — the gap between peak and achieved is the subject of Chapter 8's MFU.

Two consequences to carry forward. First, the shape of efficient code: anything expressible as a large matrix multiply is fast; anything that is a Python loop over examples, or a long chain of small operations, wastes the machine. Deep learning architectures are, to a first approximation, the family of models that consist almost entirely of large matrix multiplies — the transformer most of all, which is why Chapter 3 will describe it as four matrices here, three matrices there. Second, a caveat that Chapter 8 turns into a whole framework: peak FLOP/s only materialize if data arrives fast enough. Moving bytes from memory to the compute units is often the real bottleneck, a property measured as *arithmetic intensity*, and it is why some operations (attention over long sequences, small batches at inference) run far below the numbers in Figure 1.5. For now, the slogan version suffices: *make it a big matmul, and keep it fed*.

1.6 Practical considerations

The recipe is four lines; the bugs live in the details around them. What follows is the collection of surprises that catch engineers coming from ordinary software, starting with the ones our own rent example quietly demonstrated.

Learning rate and the scale of your inputs

Look back at Table 1.1 and you will notice we used $\eta = 5 \times 10^{-5}$, an oddly small number. It was not a free choice. The gradient with respect to w contains a factor of x , and our inputs are around 100, so the gradients came out around half a million; a learning rate of 0.01 would have multiplied that into a step of thousands, overshooting the bowl entirely — the loss would have *grown* each step, doubling and doubling until it reached infinity. That is divergence, and you will meet it on real runs as a loss curve that suddenly bends upward and explodes. There is a second, quieter symptom in the same table: b barely moved in 2,000 steps. Its gradient lacks the factor of 100 that w 's has, so at a learning rate sized for w , the intercept learns 100 $\times$ slower. One knob, two parameters with gradients of wildly different scale: that tension is why practitioners normalize inputs to roughly unit scale, and, at the scale of this book, why per-parameter adaptive optimizers like AdamW (Chapter 17) and normalization layers inside the model (Chapter 3) exist at

all. The learning rate remains the single most consequential hyperparameter in deep learning; when Part III discusses schedules — warm it up, hold, decay — it is this same knob being managed over a multi-month run.

Numerical reality: floats are not numbers

Training arithmetic runs in floating point, usually 32-bit and increasingly 16-bit (Chapter 15), and floats break the algebra you learned. $0.1 + 0.2 \neq 0.3$; summation order changes results; and the two failure modes that matter are overflow (a number exceeds the format's range and becomes `inf`) and underflow to zero. The classic overflow is the softmax: it exponentiates its inputs, and e^{89} already exceeds float32's range, so every serious implementation subtracts the maximum before exponentiating — mathematically a no-op, numerically the difference between an answer and a screen of NaNs. The classic silent killer is mixing precisions carelessly: in 16-bit formats, small gradient values underflow to zero and learning quietly stops in parts of the model. One NaN anywhere poisons everything it touches, and since the loss aggregates the whole model, a single bad division at step 40,000 shows up as `loss: nan` and nothing else; Chapter 18 covers the defensive tooling. For now, adopt the habit: when a loss misbehaves, suspect numerics before logic.

Reproducibility

Training is randomized on purpose — initial parameters are random, data order is shuffled — so two runs of the same code produce different models unless you pin every seed (Python's, numpy's, the framework's). Even then, GPU kernels may sum in non-deterministic order, so bit-identical reruns cost extra and are usually reserved for debugging. The practical standard is: fix seeds, log everything (loss curves, learning rate, data position, code version), and checkpoint often, so any anomaly can be replayed from just before it happened. On a month-long run, the ability to rewind to step 61,000 and replay with more logging is not a luxury; it is how divergences get diagnosed (Chapter 18).

Common pitfalls

Forgetting to zero the gradients.

Autograd *accumulates* gradients into `.grad` rather than overwriting — a feature you will later exploit deliberately (gradient accumulation, Chapter 4), and a bug on day one: without `optimizer.zero_grad()` each step, you descend along the *sum* of all past gradients. Symptom: training that seems to work, badly.

Shape bugs that broadcast instead of crashing.

Subtract a `(B, )` tensor from a `(B, 1)` tensor and numpy/PyTorch will happily produce `(B, B)` — no error, garbage loss. Most "my model won't learn" reports with clean-looking code are this. Defense: assert shapes at function boundaries, and distrust any loss whose value looks reasonable but never improves.

Loss reduction changing your effective learning rate.

`mean` versus `sum` over the batch differ by a factor of the batch size, and the learning rate that worked for one diverges with the other. Decide once, and re-derive when you change batch size — this exact issue returns at scale in Chapter 17.

Evaluating with the graph still on.

Running validation without `torch.no_grad()` records the autograd graph for data you will never backpropagate — memory climbs, and eval OOMs at a batch size training handled. The paired mistake is forgetting `model.eval()`, which leaves train-time behaviors (dropout, batch-norm updates) active during measurement.

Testing on what you trained on.

In small projects: an accidental overlap between train and validation splits (deduplicate first — the same item under two IDs is the classic). At LLM scale: benchmark contamination in web corpora. Same disease, same result — numbers too good to be true, models that disappoint in production. Chapters 9 and 10 both fight it.

Tuning until the validation set lies.

Hundreds of choices made by watching validation loss turn the validation set into a second training set. Keep a final test set you touch once; report that.

Reading too much into one loss number.

The loss that matters is the trend on held-out data, compared across identical data and measurement conditions. A lower training loss than yesterday's run with a different data mix is not evidence of anything.

1.7 Where this shows up later

Every chapter of this book is one of this chapter's boxes, blown up to industrial size. The loss function becomes next-token cross-entropy in Chapter 2, and that single swap is what turns this recipe into a language model; the function family becomes the transformer in Chapter 3. The four-line loop grows into Chapter 4's real training loop — batches, learning-rate schedules, checkpoints — and the machine under it stops being one chip: Part III distributes exactly this loop, unchanged in meaning, across thousands of GPUs.

The two properties of backpropagation we flagged become entire chapters: its $2\times$ compute factor is the backbone of Chapter 8's FLOP accounting (and of every cost estimate in Chapter 6), and its stored-activations memory bill is what Chapter 15's checkpointing and mixed precision exist to shrink. The learning-rate tension from Section 1.6 matures into Chapter 17 (AdamW, schedules, batch-size scaling), and the divergence we induced with a too-large step is the loss spike of Chapter 18, where it is diagnosed on real 70B runs. Generalization discipline becomes evaluation at scale in Chapter 9 — held-out perplexity, benchmarks, and the contamination problem — and the "your data is the product" observation becomes Chapter 10. When any later chapter feels abstract, the fix is to come back here and find the two-parameter version of the same idea; it always exists.

1.8 Summary

Machine learning replaces writing a function with searching for one. You choose a parameterized family, you write a loss that scores candidates against examples, and gradient descent — feel the slope, step downhill, repeat — does the

searching. Backpropagation makes the slope affordable: by running the chain rule backward over the recorded computation graph, the full gradient costs about two extra forward passes, for any number of parameters, at the price of storing the forward pass's intermediates in memory. The optimizer only ever minimizes training loss, which is a proxy; held-out data is the only honest measure of what you actually built, and protecting its honesty — splits, an untouched test set, contamination checks — is a discipline, not a formality. And the whole recipe compiles down to large matrix multiplies, which is why it runs on GPUs and why the hardware ladder from Python loops to an H100 spans seven orders of magnitude. Four lines — forward, loss, backward, update — plus honest measurement and enough matmuls per second: that is training. The other twenty-eight chapters are about doing it well.

References

1. Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. The 7B checkpoint contains 6,738,415,616 parameters (counted from the released weights; the arithmetic is done by hand in Chapter 3).
2. Robbins, H., & Monro, S. (1951). A Stochastic Approximation Method. *Annals of Mathematical Statistics*, 22(3), 400–407. The origin of stochastic gradient methods; every optimizer in this book descends from this paper.
3. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning Representations by Back-propagating Errors. *Nature*, 323, 533–536. Backpropagation as the practical training algorithm for multi-layer networks; the $\sim 2\times$ forward cost of the reverse pass is standard in the automatic-differentiation literature, e.g. Baydin et al.⁴
4. Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic Differentiation in Machine Learning: a Survey. *JMLR*, 18(153), 1–43. arXiv:1502.05767. Reverse-mode autodiff computes gradients at a small constant multiple of the cost of the function evaluation, independent of the number of inputs.
5. Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2017). Understanding Deep Learning Requires Rethinking Generalization. *ICLR*. arXiv:1611.03530. Standard architectures reach zero training error on ImageNet and CIFAR-10 with completely randomized labels.
6. NVIDIA (2020). NVIDIA A100 Tensor Core GPU Datasheet. Peak dense BF16 throughput 312 TFLOP/s.
7. NVIDIA (2022). NVIDIA H100 Tensor Core GPU Datasheet. Peak dense BF16 throughput 989 TFLOP/s (1,979 TFLOP/s with structured sparsity; dense figure used here).